Program Studi Teknik Informatika
Institut Teknologi Sumatera

IF25-11001 ALGORITMA PEMROGRAMAN

Modul Belajar 11

Searching

Dari 40 nama di daftar hadir sampai kamus yang bisa dicari dalam beberapa langkah

Tim Pengajar

Algoritma Pemrograman. Tahun 2026

Komponen	Keterangan
Pertemuan	11 dari 16
Capaian	Sub-CPMK 11.1 dan 11.2, turunan CPMK 0607 dan 0608
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 100 menit di kelas
Latihan	`02 Latihan/P11 - Searching - Latihan/`

Peta belajar

Bagian	Isi
0	Tentang modul ini dan cara memakainya
1	Masalah pembuka
Langkah 1	linear search
Langkah 2	trace linear search
Langkah 3	kasus terbaik dan terburuk
Langkah 4	binary search
Langkah 5	trace low, high, dan mid
Langkah 6	binary search saat target tidak ada
Langkah 7	membandingkan kedua algoritma
Langkah 8	dua kesalahan binary search
Penutup	Latihan mandiri, bawa ke tugas, rangkuman, cek pemahaman, bacaan lanjut

Tentang modul ini

Modul ini menemani Pertemuan 11. Isinya sama dengan slide, tetapi ditulis lebih pelan supaya bisa dipelajari sendiri, termasuk oleh yang berhalangan hadir.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Diukur di
11.1	Merancang pseudocode linear search (termasuk hasil tidak ditemukan dan semua kemunculan) dan binary search pada data terurut	Exit ticket soal 1, Tugas 11, PBL 2, UAS
11.2	Menelusuri binary search dengan trace table low, high, mid, membandingkan banyak perbandingan kedua algoritma, dan menjelaskan syarat data terurut	Exit ticket soal 2, Tugas 11, UAS

Keduanya menurunkan CPMK 0607 dan 0608: menerapkan (0607) dan menjelaskan (0608) konsep dasar algoritma dan sistem komputer untuk menyelesaikan masalah komputasi.

Cara memakai modul ini

Setiap langkah punya kotak merah **Tebak dulu**. Tutup kotak hijau **Jawaban** di bawahnya dengan kertas, tulis dugaan kalian, baru buka. Prosedur penting dijabarkan dalam tiga kolom berdampingan yang dibaca dari kiri ke kanan; sebelum membaca kolom berikutnya, coba tebak dulu langkah apa yang akan datang. Rangkaian setengah jadi lalu mandiri mengurangi bantuan sedikit demi sedikit.

Yang perlu disiapkan

Kertas bergaris, pensil, dan nilai *A* dan *B* dari NIM kalian: *A* digit terakhir (kalau 0, pakai 5) dan *B* digit kedua dari belakang (kalau 0, pakai 3).

Masalah pembuka

PEMBUKA

Asisten mencari nama Rizky di daftar hadir 40 orang yang ditulis sesuai urutan datang. Satu-satunya cara adalah membaca dari atas, satu per satu. Bandingkan dengan mencari kata di kamus tebal: tidak ada yang mulai dari halaman pertama. Kita membuka di tengah, melihat apakah kata yang dicari ada di depan atau di belakang, lalu membuang separuh yang lain. Cara kedua jauh lebih cepat, tetapi hanya mungkin karena kamus terurut. Pertemuan ini membahas kedua cara itu.

Langkah 1: linear search

LINEAR SEARCH

Linear search membandingkan setiap elemen dengan target, mulai dari indeks 0, dan berhenti begitu menemukannya. Hasilnya adalah indeks elemen itu, atau −1 jika data habis tanpa menemukan. Nilai −1 dipilih karena indeks sah selalu 0 atau lebih. Algoritma ini bekerja pada data yang tidak terurut.

```
idx ← -1
i ← 0
SELAMA i < N DAN idx = -1
  JIKA arr[i] = target MAKA
    idx ← i
  AKHIR JIKA
  i ← i + 1
AKHIR SELAMA
OUTPUT idx // -1 berarti tidak ada
```

Langkah 2: trace linear search

LINEAR SEARCH

Untuk `arr` = 12, 5, 18, 3, 9, 15, 1 dan target 9, perbandingan dilakukan dengan 12, 5, 18, 3, lalu 9. Target ditemukan di indeks 4 setelah lima perbandingan. Secara umum, target di indeks k ditemukan setelah $k + 1$ perbandingan.

Langkah 3: kasus terbaik dan terburuk

LINEAR SEARCH

Kasus terburuk linear search terjadi saat target tidak ada atau berada di akhir: semua n elemen diperiksa.

TEBAK DULU

Pada data yang sama, cari 7. Berapa perbandingan sebelum algoritma yakin 7 tidak ada? Dan berapa jika targetnya 12?

JAWABAN

Untuk 7, tujuh perbandingan: semua elemen harus diperiksa. Untuk 12, satu perbandingan. Kasus terburuk n perbandingan, kasus terbaik 1, ditulis $O(n)$ untuk yang terburuk.

Setengah jadi

Posisi Rizky di Rizky, Ayu, Rizky, Dita.

```
count ← ____
UNTUK i ← 0 SAMPAI ____
  JIKA nama[i] = "Rizky" MAKA
    OUTPUT ____
    count ← ____
  AKHIR JIKA
AKHIR UNTUK
```

Mandiri

Tulis pseudocode yang mencari posisi **terakhir** nilai di atas 80 dalam `nilai[N]`, atau −1. Bisakah kalian berhenti lebih awal?

JAWABAN

Setengah jadi: 0, `N-1`, `i`, `count + 1`; tercetak 0 dan 2. Mandiri: telusuri dari `N-1` turun dan berhenti di temuan pertama.

UJI DIRI

Pilih jawaban yang benar. (A) 2, (B) 0, (C) −1, (D) 4.

JAWABAN: A

Tanpa syarat berhenti, temuan kedua menimpa temuan pertama, jadi idx berisi posisi terakhir. Pilihan B benar hanya untuk versi yang berhenti. Pilihan D mengembalikan nilai, bukan indeks.

Langkah 4: binary search

BINARY SEARCH

Binary search bekerja seperti mencari kata di kamus. Variabel `low` dan `high` membatasi rentang yang masih mungkin berisi target, dan `mid` adalah indeks tengahnya. Jika `arr[mid]` lebih kecil dari target, target pasti di kanan, sehingga `low` menjadi `mid + 1`. Jika lebih besar, `high` menjadi `mid - 1`. Perulangan berhenti saat target ditemukan atau saat `low` melewati `high`. Semua ini hanya sah jika data terurut.

```
low ← 0
high ← N - 1
idx ← -1
SELAMA low ≤ high DAN idx = -1
  mid ← (low + high) DIV 2
  JIKA arr[mid] = target MAKA
    idx ← mid
  SELAINNYA JIKA arr[mid] < target MAKA
    low ← mid + 1
  SELAINNYA
    high ← mid - 1
  AKHIR JIKA
AKHIR SELAMA
```

Langkah 5: trace low, high, dan mid

BINARY SEARCH

Untuk `arr` = 3, 5, 9, 12, 17, 21, 25, 30, 34, 40 dan target 34, langkah pertama memeriksa `mid` 4 (nilai 17, lebih kecil) sehingga `low` menjadi 5. Langkah kedua memeriksa `mid` 7 (nilai 30) sehingga `low` menjadi 8. Langkah ketiga memeriksa `mid` 8 dan menemukan 34. Tiga perbandingan, sedangkan linear search butuh sembilan.

Langkah 6: binary search saat target tidak ada

BINARY SEARCH

Saat target tidak ada, rentang terus menyempit sampai kosong. Posisi akhir `low` justru menunjukkan tempat target seharusnya disisipkan.

TEBAK DULU

Pada data yang sama, cari 10. Tulis low, high, dan mid di setiap langkah. Kapan berhenti?

JAWABAN

(0,9,4) nilai 17, high 3. (0,3,1) nilai 5, low 2. (2,3,2) nilai 9, low 3. (3,3,3) nilai 12, high 2. Sekarang low 3 melewati high 2: berhenti dengan **−1** setelah empat perbandingan. Target 10 seharusnya berada di antara 9 dan 12.

Langkah 7: membandingkan kedua algoritma

BINARY SEARCH

Linear search butuh paling banyak n perbandingan, ditulis $O(n)$. Binary search membuang separuh rentang di setiap langkah, sehingga butuh paling banyak sekitar $\log_2 n$ perbandingan, ditulis $O(\log n)$. Untuk sejuta data, bedanya sejuta lawan dua puluh. Harga yang dibayar adalah syarat data terurut.

UJI DIRI

Pilih jawaban yang benar. (A) **−1**, tidak ditemukan, (B) 3, (C) 0, (D) tidak pernah berhenti.

JAWABAN: A

mid 2 bernilai 22, lebih besar, high 1. mid 0 bernilai 15, lebih besar, high **−1**. Rentang habis. Angka 8 ada di indeks 3, tetapi algoritma membuang bagian yang salah karena datanya tidak terurut.

UJI DIRI

Pilih jawaban yang benar. (A) 5, (B) 31, (C) 16, (D) 15.

JAWABAN: A

Rentang 31, 15, 7, 3, 1: lima perbandingan. B adalah linear search. C dan D menganggap hanya satu kali dibagi dua.

Langkah 8: dua kesalahan binary search

BINARY SEARCH

Dua kesalahan binary search paling sering. Pertama, `low ← mid` tanpa ditambah satu: rentang bisa berhenti menyempit dan perulangan tidak pernah selesai. Kedua, syarat `low < high`: saat rentang tinggal satu elemen, elemen itu tidak pernah diperiksa.

Latihan mandiri

Kerjakan lembar latihan di folder `02 Latihan`. Urutannya disusun dari mengingat ke menganalisis.

Tingkat	Waktu	Yang dilatih
1, Dasar	5′	trace linear search dan syarat binary search
2, Menengah	8′	pencarian di array sejajar dan trace binary search
3, Tantangan	7′	memperbaiki binary search
Pengayaan	sisa	pola banyak perbandingan binary search

Kesalahan yang sering muncul

Yang ditulis	Masalahnya	Obatnya
`mid ← (0 + N - 1) DIV 2` di setiap langkah	Rentang tidak pernah menyempit	Hitung dari `low` dan `high`
`low ← mid`	Perulangan bisa tidak berhenti	`low ← mid + 1`
`SELAMA low < high`	Elemen terakhir rentang tidak diperiksa	`low ⩽ high`
Binary search pada data acak	Hasil salah tanpa peringatan	Urutkan dulu, atau pakai linear search

Bawa ke tugas

Tugas 11 dikerjakan di kertas: untuk `arr` = 12, 5, 18, 3, 9, 15, 1, trace linear search untuk 15 dan hitung perbandingannya; urutkan data itu, lalu trace binary search untuk 15 dan untuk 4; tulis pseudocode linear search yang mencari semua posisi target. Untuk PBL 2, tambahkan fitur pencarian ke Data Analyzer beserta trace table pencarian yang berhasil dan yang gagal.

Rangkuman

Periksa satu per satu. Tidak perlu terurut. Terburuk n perbandingan, $O(n)$. Hasil −1 jika tidak ada. Buka tengah, buang separuh: `low ← mid + 1` atau `high ← mid - 1`, selama `low ⩽ high`. $O(\log n)$. Binary search hanya benar pada data terurut. Pada data acak hasilnya salah tanpa peringatan.

Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat jauh lebih menguatkan ingatan daripada membaca ulang.

No	Soal
1	Linear search pada 50 data, target tidak ada. Berapa perbandingan?
2	Apa syarat binary search?
3	Binary search pada 1.000 data, paling banyak berapa perbandingan?
4	Mengapa `low ← mid + 1`, bukan `low ← mid`?
5	Apa arti hasil **−1**?

KUNCI CEK PEMAHAMAN

(1) 50. (2) Data terurut. (3) Sekitar 10. (4) `arr[mid]` sudah diperiksa; tanpa +1 rentang bisa tidak menyempit. (5) Target tidak ditemukan; indeks sah tidak pernah negatif.

Bacaan lanjut

Gaddis, T. *Starting Out with C++*, Pearson. Deitel, P. dan Deitel, H. *C++ How to Program*, Pearson. Cormen dkk. *Introduction to Algorithms*, MIT Press. Visualisasi interaktif di `visualgo.net`.